Phage
SECURITY

— SECURITY REVIEW · PREPARED FOR

# Alt *Fun*

DATE

09.05.2026

CONDUCTED BY

Pyro, Lead Security Researcher
BengalCatBalu, Lead Security Researcher
Kvarr, Security Researcher

COMMIT

474837d

REMEDIATION

93d70e8

## Table of Contents

## About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to Pyro.

## Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

## System overview

**Alt Fun** is a permissionless token launchpad on HyperEVM. Anyone can launch a new ERC20 paired with a BounceTech Leveraged Token as the curve's reserve, and users trade in USDC the whole way through.

Each token starts on an internal bonding curve and graduates to a HyperSwap V2 pool once it crosses a USD threshold (or all of the tokens are sold), with the resulting LP locked permanently. A small fee is skimmed on every buy and sell and split between the token's creator and the protocol.

### Overall Security Posture

The following sections describe the security posture of the Alt Fun protocol before and after the fix review.

The fix review is the stage of the audit where Phage Security validated the remediations implemented by the Alt Fun team for the issues reported during the audit.

### Before remediation

The codebase came in unusually clean. Only two findings reached Medium severity and the rest were Low — there were no Highs and no Criticals, and the core curve, graduation, and LP-locking logic held up well under review. Most of what the audit surfaced sat at the seams between the launchpad and the BounceTech LT it wraps, not in the core flow itself.

The findings group into three themes:

- **Cross-graduation accounting** (M-02, L-04) — when two tokens shared the same LT, residual LT from one graduation could be silently consumed by the next, and pair donation skims routed unsolicited transfers into `LPLock` (which has no rescue path).

- **Money-flow rounding in Zap** (L-02, L-05, L-07) — closing buys overshot the curve and paid an avoidable redemption fee, the min-amount check ran on gross instead of net USDC, and the post-graduation fee policy was misdocumented.
- **Smaller liveness and design notes** (M-01, L-01, L-03, L-06, L-08) — mint/redeem pause asymmetry, missing `deadline` on Zap, and a vanity probe that could OOG `setTokenImplementation`, plus two design points (graduation trigger semantics, launch-time exchange-rate snapshot) flagged for documentation.

### After remediation

**The Alt Fun team shipped code fixes for 5 of the 10 issues (M-02, L-02, L-04, L-05, L-08) and addressed the rest with explicit natspec and documentation.**

Several patches went beyond the literal bug. The cross-graduation fix introduces a `protectedLT` snapshot and a new `_sweepLTToOwner` post-bookend that also resolves the donation/skim case (M-02 and L-04 in one pass). The closing-buy fix adds a new `Router.previewBuy` view to pre-size the LT mint and refund USDC directly. The remaining acknowledged items reflect deliberate design choices that are now spelled out on-chain.

The codebase is in good shape for mainnet.

### Security Recommendations

- **Set up a bug bounty program** to incentivize ongoing third-party review beyond the audit window.
- **Active invariant monitoring with alerting** — track `FeeVault.totalAccruedCreator + protocolBalance` vs. vault USDC balance, `LPLock.locks(token).amount` vs. the actual LP balance held at the lock, and `LTRescued` event volume vs. expected concurrent-graduation flow; page on any divergence.
- **Owner key hygiene** — document multisig signing-service policies, rotation procedure, and an incident-response runbook for compromised owner keys across `Bonding`, `Zap`, `FeeVault`, and `LPLock`. The `Ownable2StepUpgradeable` two-step transfer covers fat finger errors but not key compromise.
- **Anomaly dashboards** — surface graduation rebalance utilization (Regime 3 hits per period), LP seeding off-ratio remainder, capped buy USDC refund volume, per-LT mint-pause periods, and `FeeVault.sweepDonations` activity for community/internal review.

### Technical Overview

#### Launchpad Contracts (HyperEVM / Solidity 0.8.24)

Eight contracts in scope, all upgradeable except `Pair`, `Factory`, and `Token`:

- **Bonding** — central state machine. Owns the per-token lifecycle (Curve -> Graduating -> Graduated), runs phase 1 inline at the threshold crossing buy via `_enterGraduating`, and exposes a permissionless `finalizeGraduation` for phase 2. Holds the LP-seeding logic with hostile-pre-seed defense across three pair regimes (empty / pure-donation / mint-pre-seed), the immutable `graduationThresholdUsd`, and the launch-time vanity check (5 trailing hex zeros).

- **Router** — bonding-curve AMM math (constant-product with virtual reserves). No fees here. `previewBuy` and `_computeBuy` cap the closing buy at the pair's real balance and back-calculate `amountInUsed` from the K invariant. All state mutating entry points are gated by `BONDING_ROLE`.
- **Pair** — per token bonding curve pair. Reserves are NOT sorted by address (unlike UniswapV2) — `launchedToken` is always `tokenReserve`, paired LT is always `assetReserve`. K is pinned at `mint` and only checked (never modified) in `swap`.
- **Factory** — registry of bonding curve pairs. `setRouter` is one-shot — once any pair has been created, the router cannot be rotated.
- **Token** — ERC20 launchpad token (1B fixed supply, EIP-2612 permit). Cloned via EIP-1167 minimal proxies (~1.15M -> ~280k gas per launch), implementation is `_disableInitializers`'d. Owner (`Bonding`) is the only burner.
- **Zap** — user-facing entry point. Routes USDC to tokens via LT mint/redeem and curve/V2 swap, skims fees off every trade and forwards to `FeeVault`. Permit variants wrap permit calls in `try/catch` to defuse the standard front-run DoS. Enforces `MIN_USDC_AMOUNT = $10` on `netUsdc` and `MIN_SEED_USDC = $20` on launch seed buys.
- **FeeVault** — holds creator + protocol USDC fees from allowlisted depositors. Depositors `transfer` USDC then call `accrue` — vault never pulls. Underfund check in `accrue` (vault USDC balance >= outstanding claims) bounds depositor trust to delivered funds. Permissionless `sweepDonations` routes unbacked USDC to `feeTo`.
- **LPLock** — locks LP tokens at graduation. UUPS upgradeable. No withdraw in v1. Locker allowlist is add only — removing a locker would brick in-flight graduations, so retiring one requires a UUPS upgrade.

Three privileged roles operate the launchpad:

- **Owner (multisig)** — owns `Bonding`, `Zap`, `FeeVault`, `LPLock`. Two-step (`Ownable2StepUpgradeable`). Can rotate fees, the token implementation (subject to `TOTAL_SUPPLY` consistency), the BounceTech `GlobalStorage` mirror, and the `FeeVault feeTo`. Cannot change the post-graduation venue or LP lock — those are immutable post-init; rotation requires a UUPS upgrade.
- **BONDING_ROLE** — held by `Bonding` only. Gates every router-side entry point so curve buys/sells route through the lifecycle gates and the `Bonding`-level reentrancy guard.
- **Locker** — allowlisted callers of `LPLock.recordLock`. Held by `Bonding` in v1.

#### Token Lifecycle (states)

1. `Curve` — `buy` / `sell` accept on the curve. `LAUNCH_TRADING_DELAY_BLOCKS` (3 blocks) gates non-seed buys after `launch`; the seed buy bypasses via a transient slot consumed once.
2. `Graduating` — phase 1 fired (`canGraduate` returned true and the threshold-crossing buy ran `_enterGraduating`). Trading frozen on `Bonding`. `pendingGraduation` snapshot of `(tokensForLP, ltFromPair, lpBurned, unsoldBurned)` cached.
3. `Graduated` — phase 2 (`finalizeGraduation`) seeded the V2 LP at the cached curve-close ratio, recorded the lock in `LPLock`, swept any rebalance residue to the owner, and switched trading to HyperSwap V2 via `Zap._buyOnUniswapV2` / `_sellOnUniswapV2`.

## Executive summary

### Overview

| | |
|---|---|
| Project Name | Alt Fun (Alt Fun launchpad) |
| Repository | private |
| Commit hash | 474837dbd1e508423431c816fc2a8eb817aa3e89 |
| Remediation | 93d70e83131b50a8e607a29e52565b61651bcf7e |
| Methods | Manual review |

### Timeline

| | |
|---|---|
| Audit kick-off | 03.05.2026 |
| End of audit | 06.05.2026 |
| Remediations start | 08.05.2026 |
| Remediations end | 08.05.2026 |

### Scope

packages/contracts/src/Bonding.sol

packages/contracts/src/Zap.sol

packages/contracts/src/Router.sol

packages/contracts/src/Pair.sol

packages/contracts/src/Factory.sol

packages/contracts/src/Token.sol

packages/contracts/src/FeeVault.sol

packages/contracts/src/LPLock.sol

## Findings Summary


2 Medium · 8 Low

| ID | Title | Severity | Status |
|---|---|---|---|
| [M-01] | ‘Zap.sol::buy()‘ will get DoSed when LT minting is paused, creating a sell-only market | Medium | Acknowledged |
| [M-02] | Leftover LT from one graduation can be reused in another token’s LP Pool | Medium | Fixed |
| [L-01] | Graduation only fires on buy, contradicting ”Immediate Trigger” documented behavior | Low | Acknowledged |
| [L-02] | Closing buyer pays LT redemption fee on Leverage Token redeem | Low | Fixed |
| [L-03] | Launch-Time exchange-rate snapshot permanently shapes the bonding curve | Low | Acknowledged |
| [L-04] | Donated assets in LP pair are skimmed to ‘LPLock‘ and become stuck | Low | Fixed |
| [L-05] | Buy min-amount check validates gross USDC instead of net mint input | Low | Fixed |
| [L-06] | Missing deadline on Zap trade functions | Low | Acknowledged |
| [L-07] | Fee is charged on post graduation trades | Low | Acknowledged |
| [L-08] | Vanity probe in `setTokenImplementation` will OOG and brick impl rotation | Low | Fixed |

## Findings

### Medium Severity

#### [M-01] `Zap.sol::buy()` will get DoSed when LT minting is paused, creating a sell-only market

`LeveragedToken` is only mint-pausable, while redeems cannot be paused. If an LT gets mint-paused, then any launched token using that LT would have `buy()` DoSed, because `Zap` has to call `lt.mint()`:

```
IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
```

However, `sell()` still works, since it uses `lt.redeem()`:

```
IBounceLeveragedToken(lt).redeem(address(this), ltReceived, 0);
```

This creates a one-sided market during the curve period while minting is paused, because users can sell but nobody can buy. For example, if minting gets paused, the launched token price can only move lower in LT terms, because only selling is possible. Token holders may be incentivized to sell instead of hold, because they know buys are unavailable until minting is unpaused.

Additionally, for graduated tokens, `Zap` still mints LT on `buy()` because users input USDC. So if LT minting is paused, `Zap.sol::buy()` would still revert even though the HyperSwap `TOKEN/LT` pool itself is live. Users who already have LT could still buy directly through HyperSwap outside of `Zap`, without minting LT.

##### IMPACT

During the curve period, a mint-paused LT makes the market sell-only. Users can exit, but new buyers cannot enter, so holders may rush to sell before others do.

After graduation, `Zap.sol::buy()` still revert, while users holding LT can bypass `Zap` and buy directly on HyperSwap. This creates different behavior depending on whether the user already has LT or only has USDC.

There is also another side effect during the curve period. The exact issue about graduation only being triggerable during `buy()` is described separately, but it is amplified here. If the token is close to the graduation threshold before LT minting is paused, the LT exchange rate can still increase and make `canGraduate()` return true. However, the token only enters the Graduating state inside `Bonding.sol::buy()`, after a buy is processed. Since `Zap.sol::buy()` is unavailable while LT minting is paused, the transition to Graduating can also be blocked even though the graduation criteria are already satisfied.

##### RECOMMENDATION

For the curve period, there are a few possible choices:

1. Acknowledge and document the behavior. If LT minting is paused, it may still be acceptable to let users sell and exit to USDC.
2. If LT minting is paused and buying is DoSed, also block selling.
3. Add a way for the token creator to pause sells when the LT is mint-paused. This should probably only be possible while LT minting is paused, so creators cannot pause sells arbitrarily.

After graduation, consider adding a way to buy through `Zap` with already-held LT, so users can still access the `TOKEN/LT` pool through `Zap` even when new LT minting is paused.

#### TEAM

Acknowledged via documentation in alt-fun PR #369. The team chose option 1 (document the behavior, accept the asymmetry).

### [M-02] Leftover LT from one graduation can be reused in another token's LP Pool

When `finalizeGraduation()` is called, if the HyperSwap pair already has liquidity, the flow goes through `_seedRebalancing()`. If the attacker seeded the pool making it LT-rich, Bonding swaps `token -> LT` to fix the ratio. After `addLiquidity()`, this can leave leftover LT in Bonding. This is known, since `Bonding.sol::rescueLT()` exists to rescue LT in such cases.

However, if another token using the same LT graduates later, an attacker can preseed that second pool in the opposite direction before `finalizeGraduation()`, making it token-rich. In that case, the rebalance swaps `LT -> token`. Normally this should leave leftover tokens, which are burned in `_routerDepositAndDispose()`.

The issue is that `_routerDepositAndDispose()` checks the available LT using `IERC20(lt).balanceOf(address(this))`. This assumes the LT balance belongs to the current graduation, but it also includes LT left in `Bonding` from previous graduations using the same LT.

So instead of the owner later calling `rescueLT()`, that previous leftover LT can be used as liquidity for another token's pool.

#### IMPACT

**An attacker can cause the following:**

Token A and Token B both use the same LT and are ready to graduate.

1. Create/preseed the HyperSwap pool for Token A, making it LT-rich.
2. `finalizeGraduation(A)` leaves leftover LT in `Bonding`.
3. Create/preseed the HyperSwap pool for Token B, making it token-rich.
4. `finalizeGraduation(B)` enters `_routerDepositAndDispose()`. Normally the imbalance would leave leftover Token B, which gets burned. However, because the function uses the full LT balance of `Bonding`, the leftover LT from Token A is also used and deposited into Token B's LP instead of being rescued.

This is medium severity because the attacker may not be able to directly steal the LT, but it breaks accounting between different tokens using the same LT. Token A's leftover LT is moved into Token B's liquidity, Token B gets extra liquidity, and the owner loses LT that should have been recoverable through `rescueLT()`.

#### RECOMMENDATION

In `_routerDepositAndDispose()`, any leftover LT should be moved to a dedicated recipient or accounted separately, so later graduations using the same LT cannot consume it.

#### TEAM

Fixed in alt-fun PR #371. The team went beyond the original recommendation. `finalizeGraduation` now snapshots `protectedLT = balanceOf(this) - p.ltFromPair` BEFORE any LP work, threads it through `_routerDepositAndDispose` so the deposit allowance is sized off `(ltBal - protectedLT)`. The new `_sweepLTToOwner` helper transfers any remaining residue to the owner with an `LTRescued` event, so honest graduations no longer accumulate stuck LT either.

## Low Severity

### [L-01] Graduation only fires on buy, contradicting "Immediate Trigger" documented behavior

`docs/contracts-scope.md:73` states graduation is "executed in `Bonding._graduate()` immediately when true". The implementation only enters phase 1 from inside `_executeBuy` — `_enterGraduating` is `internal` with no external entry point:

```
// Bonding.sol — only call site
if (canGraduate(tokenAddress)) {
   _enterGraduating(tokenAddress);
}
```

`canGraduate`'s USD path reads live `exchangeRate()`, so a token can become ripe purely from a rate move. With no buy, the ripe state can be lost before it's captured (rate retrace, or a sell pulling `realLtRaised` back below threshold).

#### IMPACT

Tokens that cross the USD threshold without an accompanying buy may never graduate. Behavior diverges from documented "immediate" trigger.

#### RECOMMENDATION

Either fix the docs to say phase 1 only fires on the threshold-crossing buy, or expose a permissionless poke:

```
function pokeGraduate(address token) external {
   if (!canGraduate(token)) revert NotGraduatable();
   _enterGraduating(token);
}
```

#### TEAM

Acknowledged via documentation in alt-fun PR #367. The team opted to keep the contract surface unchanged and update the docs to clarify graduation only fires on the threshold crossing buy. A keeper bot will trigger immediate graduations off-chain via a small buy when the USD-only threshold is crossed without organic flow, so end-user behavior matches the original "immediate" intent without adding a permissionless on-chain entry point.

### [L-02] Closing buyer pays LT redemption fee on Leverage Token redeem

`Zap._executeBuy` mints LT from the full `netUsdc` before knowing how much the curve will actually accept. On a capped buy (graduating tx, when `_computeBuy` hits `tokensOut > realBalance`), the left-over LT is redeemed back to USDC for the user — and `LT.redeem` charges `percentFee = baseAmount * redemptionFee * targetLeverage` on that leg:

```
// Zap.sol — _executeBuy
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
// ... curve consumes only amountInUsed, leaving ltLeft
uint256 ltLeft = ltMinted - amountInUsed;
if (ltLeft > 0) {
   IBounceLeveragedToken(lt).redeem(msg.sender, ltLeft, 0); // pays
      redemptionFee
}
```

The round-trip is avoidable — `Router._computeBuy` is a `view` and returns the exact `amountInUsed` for any input. Zap could pre-size the mint to `ltToBaseAmount(amountInUsed)` and refund the unconverted USDC directly, leaving no LT to redeem and no fee to pay.

#### IMPACT

The closing buyer of every graduation pays an avoidable LT redemption fee on the overshoot.

#### RECOMMENDATION

Pre-size the mint against `_computeBuy` instead of overshoot-and-refund:

```
- uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
- (tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltMinted);
- uint256 ltLeft = ltMinted - amountInUsed;
- if (ltLeft > 0) IBounceLeveragedToken(lt).redeem(msg.sender, ltLeft, 0);
+ uint256 ltTarget = IBounceLeveragedToken(lt).baseToLtAmount(netUsdc);
+ (uint256 ltNeeded,) = router.previewBuy(tokenAddress, ltTarget);
+ uint256 baseToConvert = IBounceLeveragedToken(lt).ltToBaseAmount(ltNeeded);
+ IBounceLeveragedToken(lt).mint(address(this), baseToConvert, 0);
+ (tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltNeeded);
+ uint256 usdcLeft = netUsdc - baseToConvert;
+ if (usdcLeft > 0) $.usdc.safeTransfer(msg.sender, usdcLeft);
```

Refund the unused USDC directly so the closing buyer avoids the redemption-fee surcharge entirely.

#### TEAM

Fixed in alt-fun PR #370. A new public `Router.previewBuy` view was added (returning `(amountInUsed, tokensOut)` for any LT-in input), and `Zap._executeBuy` now pre-sizes the LT mint to the exact needed amount via `previewBuy(ltIfFull)` and `ltToBaseAmount(ltNeeded)`. Any unconverted USDC is refunded directly to the buyer instead of being round tripped through `LT.redeem`, so the closing buyer no longer pays the `baseAmount × redemptionFee × targetLeverage` surcharge on the overshoot. The fee proration was also tightened to charge only on the actual `baseToConvert` portion, with the unused fee refunded.

### [L-03] Launch-Time exchange-rate snapshot permanently shapes the bonding curve

`Bonding._deployAndSeed` derives `virtualLtReserve` from a single read of `LT.exchangeRate()`, and `Pair.mint` bakes it into `k = TOTAL_SUPPLY * virtualLtReserve` for the lifetime of the curve:

```
// Bonding.sol — _deployAndSeed
uint256 exchangeRate = IBounceLeveragedToken(ltAddress).exchangeRate();
uint256 virtualLtReserve = (VIRTUAL_LIQUIDITY_USD * 1e18) / exchangeRate;
$.router.addInitialLiquidity(tokenAddr, totalSupply, curveSupply,
   virtualLtReserve);
```

Higher snapshot → lower `virtualLtReserve` → lower `k` → larger price impact per LT in. Lower snapshot → higher `k`, the opposite. The creator picks `ltAddress` but has no parameter to bound the rate they accept, and `exchangeRate` is donation-sensitive (`baseAssetBalance` counts direct USDC transfers). Whatever transient state the LT is in at the inclusion block — donation, bridging, PnL spike — defines the curve forever.

#### IMPACT

Every future buy/sell price along the curve is anchored to a single externally-mutable read taken at launch. If the K parameter is set to incorrect values, it directly affects the token's future pricing.

#### RECOMMENDATION

Let the creator pass a min/max band for the accepted rate, mirroring AMM slippage:

```
 struct LaunchParams {
    ...
+    uint256 minExchangeRate;
+    uint256 maxExchangeRate;
 }
```

```
 uint256 exchangeRate = IBounceLeveragedToken(ltAddress).exchangeRate();
+ if (exchangeRate < params.minExchangeRate || exchangeRate >
   params.maxExchangeRate) {
+    revert ExchangeRateOutOfBand();
+ }
 uint256 virtualLtReserve = (VIRTUAL_LIQUIDITY_USD * 1e18) / exchangeRate;
```

#### TEAM

Acknowledged via documentation in alt-fun PR #372. The team rejected the (`min`, `max`) band proposal, as it would introduce a launch failure mode users can't see and forces the frontend into a tolerance that's either too tight (legitimate launches fail) or too loose (decorative).

### [L-04] Donated assets in LP pair are skimmed to LPLock and become stuck

When finalizing a graduation, `_seedUniswapV2Direct()` first calls `IUniswapV2Pair(pair).skim(_s().lpLock)`. This means that if the HyperSwap pair has any donated balances above reserves, they are skimmed to `LPLock`.

This is incorrect for both assets.

1. If the skimmed asset is LT, it gets sent to `LPLock` and is effectively stuck there. `LPLock` has no rescue functionality, even though the LT has actual value.
2. If the skimmed asset is the launched token, it also gets stuck in `LPLock` instead of being burned. This is inconsistent with the rest of the graduation flow, where unused tokens are burned. For example, `_routerDepositAndDispose()` burns `leftoverToken` after the router liquidity deposit.

Additionally, if the LP pool already has reserves, the flow later goes through `_seedRebalancing()` and `_routerDepositAndDispose()`. In that case, the skimmed assets could theoretically be kept in `Bonding` and used in the liquidity deposit, with any leftover TOKEN burned and any leftover LT left available for `rescueLT()`.

#### IMPACT

Donated LT can become permanently stuck in `LPLock` instead of being used or rescued. Donated launched tokens can also become stuck in `LPLock` instead of being burned, leaving supply that the rest of the graduation flow would normally remove.

#### RECOMMENDATION

Do not skim donated balances to `LPLock`. Skim them to `Bonding` instead, then let `_routerDepositAndDispose()` handle them: use what can be used for liquidity, burn leftover TOKEN, and leave leftover LT available for `rescueLT()`.

#### TEAM

Fixed in alt-fun PR #381 with the recommended approach. `_seedUniswapV2Direct` now skims donations to `address(this)` instead of `LPLock`.

### [L-05] Buy min-amount check validates gross USDC instead of net mint input

`MIN_USDC_AMOUNT = 10e6` is pre-checked on the buy path so users see `BelowMinAmount` instead of the LT's undecodable `0x05eb05ac` (`BelowMinTransactionSize`) selector. The buy path checks the wrong value: `_buyInternal` validates the gross `usdcAmount`, but `_executeBuy` deducts the fee and forwards `netUsdc` to `LeverageToken.mint`, which is what the LT actually validates against its floor.

```
// Zap.sol — _buyInternal
if (usdcAmount < MIN_USDC_AMOUNT) revert BelowMinAmount();
```

```
// Zap.sol — _executeBuy
uint256 netUsdc = usdcAmount - feeOnGross;
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
// ^ netUsdc, not usdcAmount, is what hits the LT floor
```

Any `usdcAmount` in `[MIN_USDC_AMOUNT, MIN_USDC_AMOUNT / (1 - buyFeeBps/BPS_DENOM)]` passes the Zap check and reverts inside `mint` with the exact selector the pre-check was added to suppress.

RECOMMENDATION

Validate the value forwarded to `mint`:

```
  uint256 netUsdc = usdcAmount - feeOnGross;
+ if (netUsdc < MIN_USDC_AMOUNT) revert BelowMinAmount();
  $.usdc.forceApprove(lt, netUsdc);
  uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
```

Alternatively, rework the money flow and deduct fees after calling `lt.mint`.

TEAM

Fixed in alt-fun PR #378 with the recommended approach.

### [L-06] Missing deadline on Zap trade functions

Both buy and sell functions inside `Zap.sol` accept a `deadline` parameter.

`minTokensOut` / `minUsdcOut` bound the magnitude of the loss, but not the time window in which an MEV can realize it.

Example (using simple values):

1. User submits `buy(token, 100 USDC, minTokensOut=95)`, current rate is 1:1
2. TX is delayed a bit (very low likelihood, but possible)
3. Curve rate falls (e.g. 1.0 → 0.5). Same 100 USDC now mints 200 curve tokens
4. User's TX executes
5. MEV bot sees the stale TX in the mempool and sandwiches it (buy to push the curve back to 1:1 + sell back to 1:0.5)

User got their 5% slippage against the original rate, against the executed rate they paid ~50% over fair value. Same can happen with `sell`.

HyperEVM has MEV, however the potential for this attack is small due most TX being included in the block that they are made.

RECOMMENDATION

Consider adding a `deadline` to `buy/sell` (and their permit versions):

```
  function buy(
     address tokenAddress,
     uint256 usdcAmount,
     uint256 minTokensOut,
     address referrer,
+    uint256 deadline
  ) external nonReentrant returns (uint256 tokensOut) {
+    if (block.timestamp > deadline) revert DeadlineExpired();
     return _buyInternal(tokenAddress, usdcAmount, minTokensOut, referrer);
  }
```

TEAM

Acknowledged. The team noted that HyperEVM's typical inclusion behavior keeps the practical attack window narrow, so the `deadline` parameter was not added in this remediation round.

### [L-07] Fee is charged on post graduation trades

The provided docs show this table

| Fee | Rate | Split |
|---|---|---|
| Router buy | 0.5% | 0.4% protocol / 0.1% creator |
| Router sell | 0.5% | 0.4% protocol / 0.1% creator |
| HyperSwap swap (post-grad) | 0.3% | HyperSwap LPs |
| LT redemption | BounceTech-internal | No Alt Fun fee |

Where we can see that after graduation there shouldn't be a fee on swaps. However the still code charges `buyFeeBps` / `sellFeeBps`:

```
if ($.bonding.isGraduated(tokenAddress)) {
   tokensOut = _buyOnUniswapV2(tokenAddress, lt, ltMinted);
   amountInUsed = ltMinted;
} else {
   (tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltMinted);
}
actualFee = ltMinted == 0
   ? 0
   : (usdcAmount * buyFeeBps_ * amountInUsed) / (BPS_DENOM * ltMinted);
```

RECOMMENDATION

Change the code to use 0 fee when the token has graduated

```
// Zap._executeBuy
- uint256 buyFeeBps_ = $.buyFeeBps;
+ uint256 buyFeeBps_ = $.bonding.isGraduated(tokenAddress) ? 0 : $.buyFeeBps;
  uint256 feeOnGross = (usdcAmount * buyFeeBps_) / BPS_DENOM;

// Zap._sellInternal
- uint256 fee = (grossUsdc * $.sellFeeBps) / BPS_DENOM;
+ uint256 sellFeeBps_ = bonding_.isGraduated(tokenAddress) ? 0 : $.sellFeeBps;
+ uint256 fee = (grossUsdc * sellFeeBps_) / BPS_DENOM;
```

TEAM

Acknowledged via documentation in alt-fun PR #383. The team confirmed the original docs were misleading: charging fees on every venue (curve and post-graduation) is the intended policy — lift-

ing fees post-graduation would silently halve protocol+creator revenue the moment a token graduated, which is the opposite of the intent.

### [L-08] Vanity probe in `setTokenImplementation` will OOG and brick impl rotation

`setTokenImplementation` runs `VanityMining.mine` as a sanity probe before swapping the impl. The miner brute forces a salt with 20 zero trailing bits (~1M iters average per the lib's own comment).

```
VanityMining.mine(address(0x1), bytes32(0), bytes32(0), newImpl, address(this),
   0);
```

Per loop cost is about 150 gas. With hype's big block caps at 30M gas, it means that the probe must converge in <200k loops to fit, so ~83% of impls OOG.

Since the salt sequence is deterministic for `(0x1, 0, 0, newImpl, this, 0)`, retrying the same `newImpl` always OOGs at the same iter, which means that the owner has to redeploy `Token` to fresh addresses until one happens to converge under the cap.

#### RECOMMENDATION

Consider removing the `VanityMining` part:

```
-   VanityMining.mine(address(0x1), bytes32(0), bytes32(0), newImpl, address(
   this), 0);
    address old = $.tokenImplementation;
    $.tokenImplementation = newImpl;
```

#### TEAM

Fixed in alt-fun PR #385 with the recommended approach. The `VanityMining.mine` probe was removed from `setTokenImplementation` outright. The function now updates `tokenImplementation` directly after the existing `TOTAL_SUPPLY` consistency check.